

SKILL WEEK-4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Handle empty command
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }

    // Tokenize input
    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count++] = token;
        token = strtok(NULL, " \t");
    }

    printf("\n--- Parsing Result ---\n");

    for (int i = 0; i < count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }
}
```

```

// Tokenize input
char *token = strtok(input, " \t");

while (token != NULL && count < MAX_TOKENS) {
    tokens[count++] = token;
    token = strtok(NULL, " \t");
}

printf("\n--- Parsing Result ---\n");

for (int i = 0; i < count; i++) {
    printf("Argument %d: %s\n", i + 1, tokens[i]);
}

printf("Total tokens: %d\n", count);

return 0;
}
|

```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```

laasya@laasya:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

laasya@laasya:~$ mkdir -p ~/skill_week4
laasya@laasya:~$ cd ~/skill_week4
laasya@laasya:~/skill_week4$ nano task1.c
laasya@laasya:~/skill_week4$ gcc task1.c -o task1
laasya@laasya:~/skill_week4$ ./task1
myShell> echo hello

--- Parsing Result ---
Argument 1: echo
Argument 2: hello
Total tokens: 2
laasya@laasya:~/skill_week4$ |

```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```

#include <stdio.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;
    int in_single_quote = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\') {
            in_single_quote = !in_single_quote;
        }
    }

    if (in_single_quote) {
        printf("Syntax Error: Unmatched single quote.\n");
        return 1;
    }

    char *p = input;

    while (*p != '\0' && count < MAX_TOKENS) {
        while (*p == ' ' || *p == '\t')
            p++;

        if (*p == '\0')
            break;
    }
}

```

```

    if (*p == '\\') {
        p++;

        tokens[count++] = p;

        while (*p != '\\' && *p != '\\0')
            p++;

        if (*p == '\\') {
            *p = '\\0';
            p++;
        }
    } else {
        tokens[count++] = p;

        while (*p != ' ' && *p != '\\t' && *p != '\\0')
            p++;

        if (*p != '\\0') {
            *p = '\\0';
            p++;
        }
    }
}

printf("\n--- Parsing Result ---\n");

for (int i = 0; i < count; i++) {
    printf("Argument %d: %s\n", i + 1, tokens[i]);
}

return 0;
}
|

```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify

```

laasya@laasya:~/skill_week4$ nano task1.c
laasya@laasya:~/skill_week4$ nano task2.c
laasya@laasya:~/skill_week4$ gcc task2.c -o task2
laasya@laasya:~/skill_week4$ ./task2
myShell> echo hello

--- Parsing Result ---
Argument 1: echo
Argument 2: hello
laasya@laasya:~/skill_week4$ echo 'hello world'
hello world

```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    int in_double_quote = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '"') {
            in_double_quote = !in_double_quote;
        }
    }

    if (in_double_quote) {
        printf("Syntax Error: Unmatched double quote.\n");
        return 1;
    }

    char *p = input;

    while (*p != '\0' && count < MAX_TOKENS) {
        while (*p == ' ' || *p == '\t')
            p++;
    }
}

```

```

    if (*p == '\\0')
        break;

    if (*p == '"') {
        p++;

        tokens[count++] = p;

        while (*p != '"' && *p != '\\0')
            p++;

        if (*p == '"') {
            *p = '\\0';
            p++;
        }
    } else {
        tokens[count++] = p;

        while (*p != ' ' && *p != '\\t' && *p != '\\0')
            p++;

        if (*p != '\\0') {
            *p = '\\0';
            p++;
        }
    }
}

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < count; i++) {

    char *user_pos = strstr(tokens[i], "$USER");

    if (user_pos != NULL) {
        char result[MAX_INPUT];

```

```

    if (user_pos != NULL) {
        char result[MAX_INPUT];
        char *username = getenv("USER");

        if (username == NULL)
            username = "unknown";

        int prefix_length = user_pos - tokens[i];

        snprintf(result, sizeof(result),
                 "%.*s%s",
                 prefix_length,
                 tokens[i],
                 username,
                 user_pos + 5);

        printf("Argument %d: %s\n", i + 1, result);
    } else {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }
}

return 0;
}
|

```

^{^G} Help ^{^O} Write Out ^{^W} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^_} Replace ^{^U} Paste ^{^J} Justify

```

-bash: /home/laasya/skill_week4: Is a directory
laasya@laasya:~$ pwd skill_week4
/home/laasya
laasya@laasya:~$ cd skill_week4
laasya@laasya:~/skill_week4$ nano task3.c
laasya@laasya:~/skill_week4$ gcc task3.c -o task3
laasya@laasya:~/skill_week4$ ./task3
myShell> echo "hello $user"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: hello $user
laasya@laasya:~/skill_week4$ |

```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    char *p = input;

    while (*p != '\0' && count < MAX_TOKENS) {
        while (*p == ' ' || *p == '\t')
            p++;

        if (*p == '\0')
            break;

        tokens[count++] = p;

        char *write = p;

        while (*p != '\0') {
            if (*p == '\\') {
                p++;

                if (*p != '\0') {
                    *write++ = *p;
                }
            }
        }
    }
}
```

```

        *write++ = *p;
        p++;
    }
}
else if (*p == ' ' || *p == '\\t') {
    break;
}
else {
    *write++ = *p;
    p++;
}
}

*write = '\\0';

if (*p != '\\0')
    p++;
}

printf("\\n--- Parsed Arguments ---\\n");

for (int i = 0; i < count; i++) {
    printf("Argument %d: %s\\n", i + 1, tokens[i]);
}

return 0;
}
|

```

 ^G Help	 ^O Write Out	 ^W Where Is	 ^K Cut	 ^T Execute
 ^X Exit	 ^R Read File	 ^\ Replace	 ^U Paste	 ^J Justify

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char *args[] = {"ls", NULL};

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process PID: %d\n", getpid());

        execvp(args[0], args);

        // This runs only if execvp fails
        perror("Execution failed");
        exit(1);
    }
    else {
        // Parent process
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }
}

```

^G Help
 ^X Exit

^O Write Out
 ^R Read File

^W Where Is
 ^\ Replace

^K Cut
 ^U Paste

^T Execute
 ^J Justify

```

if (pid == 0) {
    // Child process
    printf("Child Process PID: %d\n", getpid());

    execvp(args[0], args);

    // This runs only if execvp fails
    perror("Execution failed");
    exit(1);
}
else {
    // Parent process
    printf("Parent Process PID: %d\n", getpid());
    printf("Created Child PID: %d\n", pid);

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
}

return 0;
}

```

[^]G Help [^]O Write Out [^]W Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

```

command_executor.c practical_week2 week4 wweek4p_task2
information skill_week4 week4p_task2 wweek4p_task2.c
information.txt 'tmp information' week4p_task2.c
ossp week2 wweek4p_task1
laasya@laasya:~$ nano task5.c
laasya@laasya:~$ gcc task5.c -o task5
laasya@laasya:~$ ls
command_executor practical 'tmp information' week4p_task2.c
command_executor.c practical_week2 week2 wweek4p_task1
information skill_week4 week3 wweek4p_task1.c
information.txt task5 week4 wweek4p_task2
ossp task5.c week4p_task2 wweek4p_task2.c
laasya@laasya:~$ ./task5
Parent Process PID: 975
Created Child PID: 976
Child Process PID: 976
command_executor practical 'tmp information' week4p_task2.c
command_executor.c practical_week2 week2 wweek4p_task1
information skill_week4 week3 wweek4p_task1.c
information.txt task5 week4 wweek4p_task2
ossp task5.c week4p_task2 wweek4p_task2.c
Child exited with status: 0

```

```

#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char *args[] = {"ls", "-l", NULL};

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process PID: %d\n", getpid());

        execvp(args[0], args);

        // This runs only if execvp fails
        perror("Execution failed");
        exit(1);
    }
    else {
        // Parent process
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }
}

```

```

laasya@laasya:~$ nano task5.c
laasya@laasya:~$ gcc task5.c -o task5
laasya@laasya:~$ ./task5
Parent Process PID: 995
Created Child PID: 996
Child Process PID: 996
total 136
-rwxr-xr-x 1 laasya laasya 16368 Aug  3 05:42 command_executor
-rw-r--r-- 1 laasya laasya 1109 Aug  3 05:24 command_executor.c
prw-r--r-- 1 laasya laasya  0 Aug 29 08:23 information
-rw-r--r-- 1 laasya laasya  49 Aug 29 08:12 information.txt
drwxr-xr-x 11 laasya laasya 4096 Aug 22 16:56 ossp
drwxr-xr-x 2 laasya laasya 4096 Aug 17 04:50 practical
drwxr-xr-x 3 laasya laasya 4096 Aug 17 05:36 practical_week2
drwxr-xr-x 2 laasya laasya 4096 Aug 18 10:09 skill_week4
-rwxr-xr-x 1 laasya laasya 16272 Aug 29 10:10 task5
-rw-r--r-- 1 laasya laasya  845 Aug 29 10:10 task5.c
prw-r--r-- 1 laasya laasya  0 Aug 29 08:22 'tmp information'
drwxr-xr-x 2 laasya laasya 4096 Aug 17 04:50 week2
drwxr-xr-x 2 laasya laasya 4096 Aug 22 17:09 week3
drwxr-xr-x 2 laasya laasya 4096 Aug 17 05:52 week4
-rwxr-xr-x 1 laasya laasya 16272 Aug 24 05:04 week4p_task2
-rw-r--r-- 1 laasya laasya  621 Aug 24 05:04 week4p_task2.c
-rwxr-xr-x 1 laasya laasya 16400 Aug 24 05:00 wwек4p_task1
-rw-r--r-- 1 laasya laasya 1462 Aug 24 04:59 wwек4p_task1.c
-rwxr-xr-x 1 laasya laasya 16224 Aug 24 05:01 wwек4p_task2
-rw-r--r-- 1 laasya laasya  556 Aug 24 05:01 wwек4p_task2.c
Child exited with status: 0

```

```

#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char *args[] = {"ls", "@l", NULL};

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child Process PID: %d\n", getpid());

        execvp(args[0], args);

        // This runs only if execvp fails
        perror("Execution failed");
        exit(1);
    }
    else {
        // Parent process
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }
}

```

```

laasya@laasya:~$ nano task5.c
laasya@laasya:~$ gcc task5.c -o task5
laasya@laasya:~$ ./task5
Parent Process PID: 1011
Created Child PID: 1012
Child Process PID: 1012
ls: cannot access '@l': No such file or directory
Child exited with status: 2
laasya@laasya:~$ |

```